

A Project Report
on
Encoder-based LLMs: Building QA systems and
Comparative Analysis

Submitted for the Partial Fulfillment of the Requirements for the degree of
Bachelor of Technology

in

Computer Science and Engineering

by

Saket Chaudhari
Roll No.: UI19CO48

Under the guidance of
Ms. Shalini Dangi



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SURAT-394190
SEPTEMBER - 2023

Indian Institute of Information Technology Surat
Computer Science and Engineering Department



CERTIFICATE

This is to certify that candidate **Saket Chaudhari** bearing Roll No: **UI19CO48** of B.TECH. IV, 8th Semester has successfully carried out the work on “**Encoder-based LLMs: Building QA systems and Comparative Analysis**” for the partial fulfillment of the degree of Bachelor of Technology (B.Tech.) in September, 2023.

Faculty Supervisor: Name: Ms. Shalini Dangi Sign:.....

1. Examiner 1: Name: Ms. Shalini Dangi Sign:.....

2. Examiner 2: Name: Dr. Pradeep Kumar Roy Sign:.....

3. Examiner 3: Name: Mr. Rajesh Ahir Sign:.....

Seal of the Institute

DECLARATION

This is to certify that

- (i) This report comprises my original work towards the degree of Bachelor of Technology in Computer Science and Engineering at Indian Institute of Information Technology (IIIT) Surat and has not been submitted elsewhere for a degree.
- (ii) Due acknowledgement has been made in the text to all other material used.

A handwritten signature in black ink, appearing to read 'Saket', enclosed within a large, loopy oval stroke.

Signature of Student
Saket Chaudhari
UI19CO48

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my heartfelt gratitude to all those who contributed to the successful completion of this project.

I want to start by sincerely thanking **Dr. J.S. Bhat** for giving me this opportunity. I want to express my sincere gratitude to my supervisor, **Ms. Shalini Dangi**, for her invaluable guidance, mentorship, and unwavering support throughout the project's completion. Her expertise and insights were instrumental in shaping the project's direction and ensuring its success. My heartfelt thanks go to my family for their unwavering support, understanding, and patience during the project's development. Their belief in me served as a constant source of motivation. I also want to express my gratitude to **IIIT Surat** for accepting the project in my chosen area of interest. I gained a lot of knowledge about Natural Language Processing and Machine Learning, which will be very beneficial to me in the future.

This project represents the fruitful outcome of personal effort and the invaluable support of the individuals and organizations mentioned above. I am sincerely grateful for their contributions, which were essential in bringing this project to successful completion.

ABSTRACT

Introduction of deep learning Large Language Models(LLMs) have caused significant development in fields of Natural Language Processing(NLP) and Computer Vision(CV). This project focuses on using the Encoder-based LLMs which include BERT and its advanced variants in building of Question-Answering (QA) systems and performing a comparative analysis of their evaluation results as the first objective. The project uses BERT as the foundation model and other advanced variants of it which includes RoBERTa, DistilBERT, ALBERT and XLM-RoBERTa.

We finetune BERT on a large reading comprehension dataset using the cloud-based NVIDIA GPU Tesla T4 provided by Google Colab. The evaluated performances are then compared and inference is conducted. Inference with the help of a chart shows the unique advantages and developments of each model along with certain limitations of the model. We further build a multilingual QA system using XLM-RoBERTa and evaluate its performance on a cross-lingual QA dataset which consists of QA pairs in 11 languages. In order to augment the size of the QA system we come up with a novel idea of using a Reader-Retriever combination. Provided a given question in a specific domain, retriever works towards fetching most relevant files from the corpus and Reader which is our finetuned BERT extracts the answer from it. The system outputs multiple answers from which the answer with best score is selected and displayed.

Contents

Title Page	i
Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
List of Figures	viii
List of Acronyms	x
1 Introduction	1
1.1 Brief of the Problem	1
1.2 Study of Existing Solution	2
1.3 Problems found with Existing Solution	2
2 Literature Review	3
2.1 Pretraining of Models	4
2.2 Pretraining Approach	4
2.3 Multilingual XLM-RoBERTa	5
2.4 Information Retrieval and BM25	6
2.4.1 Information Retrieval	6
2.4.2 BM25	6
3 Proposed System and Requirements	9
3.1 Objectives to be solved	9
3.2 Assumptions and Dependencies	10
3.3 Specific Requirements	10
3.3.1 Hardware Requirements	10
3.3.2 Software Requirements	11
3.3.3 User Interface	11

3.4	Tools and Technologies involved	11
3.5	Advantages offered by proposed system	12
4	Design	13
4.1	System Diagrams	13
4.2	BERT Architecture	15
5	Implementation	17
5.1	Answer-Retrieval QA system	17
5.1.1	Preprocessing	18
5.1.2	Training	19
5.1.3	Evaluation	20
5.2	Multilingual QA system	21
5.3	Reader-Retriever QA system	22
6	Testing and Experimental Results	23
6.1	Answer-Retrieval QA system	23
6.1.1	Evaluation of Training Results	23
6.1.2	Evaluation Results	24
6.1.3	Inference	25
6.1.4	Working of Model	25
6.2	Multilingual QA system	26
6.2.1	Evaluation results on SQuAD2 and XQuAD	26
6.2.2	Working of Model	27
6.3	Reader-Retriever QA system	28
6.3.1	Domain and Text Corpus	28
6.3.2	Working of QA system	29
7	Conclusion & Future Work	31
	References	32
	Plagiarism Report	34

List of Figures

4.1	Flowchart of Answer-Retrieval QA system	13
4.2	Reader-Retriever QA system	14
4.3	Input Output representations of BERT [1]	15
4.4	Internal Architecture of BERT [2]	16
5.1	Completion of Training of BERT	20
6.1	Training learning curve of Models	24
6.2	Evaluation Chart of Models	25
6.3	Working of Answer Retrieval QA system	26
6.4	Demonstration of Hindi QA	27
6.5	Demonstration of Spanish QA	28
6.6	Demonstration of German QA	28
6.7	All answers output by the model	30
6.8	Best answer among all output answers	30

List of Acronyms

ALBERT A Lite **BERT**.

BERT Bidirectional **E**ncoder **R**epresentation of **T**ransformer.

BM25 Best Matching 25.

CPU Central **P**rocessing **U**nit.

CV Computer **V**ision.

DistilBERT Distilled version of **BERT**.

DM Dynamic **M**asking.

GPU Graphics **P**rocessing **U**nit.

IDF Inverse **D**ocument **F**requency.

LLM Large **L**anguage **M**odels.

LSTM Long **S**hort **T**erm **M**emory.

MLM Masked **L**anguage **M**odelling.

NLP Natural **L**anguage **P**rocessing.

NSP Next **S**entence **P**rediction.

QA Question-Answering.

RNN Recurrent **N**eural **N**etworks.

RoBERTa Robustly **O**ptimized **BERT** **A**pproach.

RS Relevance **S**core.

SQuAD Stanford Question Answering Dataset.

TF Term Frequency.

TLM Translation Language Modelling.

TPU Tensor Processing Unit.

XLM-RoBERTa Crosslingual Multilingual RoBERTa.

XQuAD Cross-lingual Question Answering Dataset.

Chapter 1

Introduction

Large Language Model(LLM) is a foundational deep learning model which can be used for various Natural Language Processing(NLP) and Computer Vision(CV) tasks. LLMs are based on the concept of Transfer Learning. They are first pretrained on massive amounts of unlabelled data and then can be finetuned easily with a labelled dataset on any downstream task. Encoder refers to the encoder part of the transformer architecture used by the model. In this project we use the Bidirectional Encoder Representation of Transformer(BERT) encoder model launched by Google AI as a foundation model and its variants to build Question-Answering(QA) systems. The variants to be used include DistilBERT(Distilled version of BERT) which is launched by Huggingface, ALBERT(A Lite BERT) launched by Google Research, RoBERTa(Robustly optimized BERT approach) and XLM-RoBERTa(Cross-lingual Language Model RoBERTa) launched by Facebook AI.

1.1 Brief of the Problem

The project aims to utilize the Encoder-based Large Language Models to build Answer-Retrieval Question Answering(QA) systems, evaluating their performance on the used dataset and demonstrate their working. We then do a comparative analysis of performance of different Encoder models and do inference on the basis of results. We further try to leverage the multilingual support provided by BERT variant XLM-RoBERTa which is pretrained on 2.5 TB data containing text in many languages. Pretraining helps the model have a general understanding about all languages and answer retrieval, hence it works on question answering in multiple languages without the need to train the model for every single language. Our QA system performs well on Answer Retrieval from a given context or passage but as the amount of data increases QA gives poor results or no results as BERT can accept limited input data till 512 tokens. To solve the issue we focus on data augmentation and scaling the size of our QA system. We come up with a novel idea of using a Reader-Retriever

combination to increase the size of our QA system. Provided a given question in a specific domain, retriever works towards fetching most relevant files from the corpus and Reader which is our finetuned BERT extracts the answer from it. The system outputs multiple answers from which the answer with best score is selected and displayed.

1.2 Study of Existing Solution

There exists solutions for building complex QA systems and NLP systems in general. With the introduction of Recurrent Neural Networks(RNN) it has become easier to handle sequential data and time-series data. It has an internal memory which helps it to memorize previous inputs and makes it suitable for NLP applications. It has its limitation, as a solution Long-Short Term Memory Networks(LSTMs) are introduced. LSTM is a type of RNN architecture designed to address the limitations of traditional RNNs in capturing long-term dependencies in sequential data. The key innovation in LSTM is the introduction of special memory cells and gating mechanisms that enable it to store and access information over long periods of time. Attention mechanism is introduced to solve the limitations of RNNs which include lack of parallelism and long range dependencies. The transfer learning mechanism introduced helped in reducing the computational cost of training the model from scratch and helped to transfer features from one task to another. The precursor models used before was BiDAF model and ELMo model which attained a F1-score of 62.09 and 66.251 respectively on SQuAD 2 dataset [8]. We will be using the BERT model and its variants which uses the encoder part of transformer architecture for pretraining and later fine-tuning model on task-specific dataset. F1-score of BERT on SQuAD 2 dataset is 75.43 which makes it most appropriate model for use in project [8].

1.3 Problems found with Existing Solution

In order to perform an complex NLP task having sequential data RNN and LSTM are used as state-of-art approaches. They fail to capture longer-distance context due to vanishing gradient problem and long-term dependencies problem. They are capable of processing sequential data but cannot process data in parallel due to lack of parallel computation. They do not use attention mechanism which helps in handling longer-distance context and support parallel computation by use of hardware accelerators. Further they do not use transfer learning and hence need to be trained from scratch which is time-consuming, computationally expensive and generate a large amount of carbon footprint. Transfer learning based models like ELMo, ULMFiT [1] are unidirectional in nature and have task-specific architecture. These limitations causes poor performances on various NLP tasks.

Chapter 2

Literature Review

In this chapter we mention the academic journals referred to while working on this project and the theoretical concepts learned from the journals. We have referred papers on BERT models and provided a description about their pretraining details and pretraining approach.

Paper Title	Author & Year
1. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	Devlin et al., Oct 2018
2. Attention Is All You Need	Vaswani et al., Jun 2017
3. RoBERTa: A Robustly Optimized BERT Pretraining Approach	Liu et al., Jul 2019
4. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter	Sanh et al., Oct 2019
5. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations	Lan et al., Feb 2020
6. Unsupervised Cross-lingual Representation Learning at Scale	Conneau et al., Nov 2019
7. Modern Question Answering Datasets and Benchmarks: A Survey	Zhen Wang, Jun 2022
8. The Probabilistic Relevance Framework: BM25 and Beyond	Robertson et al., Jan 2009
9. Information Retrieval: Recent Advances and Beyond	Hambarde et al., Jan 2023

Table 2.1: Literature Review

2.1 Pretraining of Models

Pretraining is a part of transfer learning, in which the model is trained on large amounts of unlabelled data in an unsupervised fashion. By following the training procedure model learns deeper language context by analysis of the data, finding features, semantic relationships and hidden patterns in the data. The main goal of pretraining is to allow the model to easily fine-tune on any NLP task with just a limited amount of labelled dataset hence, reducing the time and computational cost. Pretraining further reduces the overall cost in terms of computation infrastructure used and reduces the carbon footprint caused by the model.

The training procedure requires use of multiple hardware accelerators like Graphics Processing Units(GPUs) and Tensor Processing Units(TPUs). The most used hardware accelerators include NVIDIA 32GB V100 GPUs and 16GB V100 GPUs along with the Cloud TPUs provided by Google. The training time requires around few days to weeks for completion, depending on the hardware used for training. For instance BERT-base model is trained on 8 NVIDIA V100 GPUs for 12 days (8 * V100 * 12).

Model	BERT	ROBERTa	DistilBERT	ALBERT
Parameters	110M	125M	66M	11M
L/H/A	12/768/12	12/768/12	6/768/12	Varies
Training Data	Books Corpus + English Wikipedia =16GB	BERT + Cnews+ Openweb text+ stories =160 GB	Books Corpus + English Wikipedia =16GB	Books Corpus + English Wikipedia =16GB

Table 2.2: Pretraining information about BERT models

2.2 Pretraining Approach

BERT - Bidirectional Encoder Representations from Transformers [1]

Pretraining Approach: Masked Language Modelling(MLM), Next Sentence Prediction(NSP)

MLM is also referred to as cloze task, in MLM 15 percent of the word sequences that are given to BERT as input are randomly masked with a [MASK] token. The model has to predict the masked words based on the other unmasked words in the context. In NSP training from a monolingual corpus two sentences are chosen and the model has to predict whether the second sentence follows the first sentence. This helps in

training the model understand the relationship between two sentences.

RoBERTa - Robustly optimized BERT approach [3]

Pretraining Approach: Dynamic Masking(DM), No NSP

RoBERTa is a replication study of BERT which uses dynamic masking instead of static masking, at any training instance every sequence is masked in a different way. It removes the NSP objective from BERT and instead increases the batch size of model and reduces the number of training steps keeping the computational cost same.

DistilBERT - Distilled version of BERT [4]

Pretraining Approach: Knowledge Distillation, DM, No NSP.

It introduces knowledge distillation, DistilBERT a student model is trained to mimic the behavior of larger BERT model - teacher model. The teacher BERT model is typically a larger, well-trained model with more parameters and better performance. Further the encoder layers in the architectures are reduced from 12 to 6.

ALBERT - A Lite BERT [5]

Pretraining Approach : MLM, Parameter Reduction and NSP

ALBERT reduces the number of embedding parameters by sharing embeddings across tokens, both within a layer and across layers. In BERT, each layer has its own set of parameters, leading to a large number of parameters in deep models. ALBERT shares the parameters across layers, meaning that certain layers use the same set of parameters, reducing the model size significantly.

2.3 Multilingual XLM-RoBERTa

XLM-RoBERTa - Cross-lingual Language Model RoBERTa [6]

XLM-RoBERTa is a powerful NLP model launched by Facebook AI, which uses 2 technologies. It merges the XLM(Cross-lingual Language Modelling) pretraining along with RoBERTa architecture.

Parameters = 125M

L/H/A = 12/768/12

Pretraining data: Filtered CommonCrawl-100 corpus = 2.5 TB

Pretraining Approach: MLM and Translation Language Modeling(TLM)

MLM is also referred to as cloze task, in MLM 15 percent of the word sequences that are given to BERT as input are randomly masked with a [MASK] token. The model has to predict the masked words based on the other unmasked words in the context. In TLM parallel sequences are considered and we concatenate parallel sequences. These sequences are then masked randomly and the model attempts to predict the masked words based on context provided by any of the two languages. This encourages model

to align and map language one and language two representations.

Combining XLM and RoBERTa leverages the strength of both XLM and RoBERTa. Hence it can perform Multilingual applications across various NLP tasks as well as give good performance results.

2.4 Information Retrieval and BM25

2.4.1 Information Retrieval

Information Retrieval(IR) [11] in NLP is a process in which documents or passages are retrieved from a large text corpus. Corpus here refers to unstructured collection of textual data in any form such as txt files, articles, news and web pages. The goal of information retrieval is to retrieve relevant information on the query/question asked by the user. An IR system achieves this by indexing the documents in the corpus, and calculating a relevance score for each document. Further it ranks all the documents on the basis of relevance score and retrieves certain number documents mentioned by the user having the highest score. IR systems are widely used in Closed-Domain QA systems, Document store systems and recommendation systems.

Components of an IR system include:

1. Document Store or Corpus: The first step in building an information retrieval system is to collect a collection of documents or files regarding a selected domain. The corpus can include text, articles, books, webpages and tweets.
2. Question and Indexing: Question is an important part of the IR system as it tells about the information user needs from the domain. The goal of an IR system is to match relevant documents to the given question. Documents are preprocessed and indexed to enable effective information retrieval. This helps in quickly locating documents that contain particular terms.
3. Relevance score and Ranking: After the user submits the query, it is preprocessed and relevance scores are generated for every document by a ranking algorithm. The ranking algorithm used can be TF-IDF, BM11 or BM25. The relevance score is proportional to the relevance of passage to the user's question. The algorithm further ranks the documents according to the scores and retrieves the documents with the highest relevance score.

2.4.2 BM25

BM25 stands for Best Matching 25 [10], it is a ranking algorithm used in information retrieval systems in both open and closed domains. It is an improved version of the

BM11 algorithm. BM25 is used for calculating the relevance score of the documents in the corpus with respect to the question/query asked by the user in the specific domain of the corpus. Next it automatically ranks the documents according to their scores and fetches the documents having best scores. The number of documents to be fetched is to be mentioned by the user in the code. The main components of BM25 include Term Frequency(TF), Inverse-Document Frequency(IDF) , Document length and tuning parameters k1 and b.

The BM25 formula for calculating the relevance score (RS) of a document for a given query is as follows:

$$RS = \Sigma(((k1 + 1) * TF) / (k1 * (1 - b + b * (DL / avgDL)) + TF)) * IDF \text{ (Eq. 2.1)}$$

TF is the term frequency(term).

DL is the length of the document.

avgDL is the average document length in the corpus.

k1 and b are the tuning parameters.

IDF is the inverse document frequency(term).

Chapter 3

Proposed System and Requirements

Chapter 3 mentions the objectives of our proposed systems with the sub-topics to be covered. We further mention the assumption and dependencies, and specific requirements of our system. The requirements include hardware, software and user interface requirements. At the end we mention tools and technologies used along with the advantages offered by our proposed system.

3.1 Objectives to be solved

Following are the Objectives of the Project:

1. Answer-Retrieval QA system:

- Build a Answer-Retrieval QA system using BERT, RoBERTa, DistilBERT and ALBERT.
- Perform a comparative analysis of its performance on the mentioned models for the given task.
- Inference on the basis of analysis results.

2. Multilingual QA system:

- Build a Multilingual Answer-Retrieval QA system using XLM-RoBERTa.
- Evaluate its performance over 11 languages using XQuAD dataset.

3. Reader-Retriever QA system:

- Reader: Use the finetuned RoBERTa model of objective1.
- Retriever: Build an information retrieval system using BM25 ranking algorithm.
- Build a Reader-Retriever QA system by merging finetuned RoBERTa(reader) and BM25 algorithm(retriever)

3.2 Assumptions and Dependencies

1. Availability of adequate computation resources: The system should fulfill all the hardware and software requirements of the project.
2. Availability of hardware accelerator: The system should have availability to hardware accelerators like GPU and TPU in order to run the project.
3. Internet Connection and Speed: The system assume that user has access to stable internet connection and having a fast internet speed.
4. Training time required: Training the models requires significant amount of time, it ranges from 3-5 hours. We need to make sure that device and internet stays connected and runtime does not get disconnected due to inactivity. The loss value should be noted down at every fixed interval in order to evaluate the performance.
5. Domain Knowledge: Implementing the project requires basic understanding of Deep Learning,NLP and transfer learning concepts
6. Python: The project requires basic understanding of python programming language and its libraries.
7. Availability of the dataset: The dataset used should be of good quality and error free. Errors in input data affects models performance.
8. Storage space: The system requires sufficient disk space to store trained models, local variables and files.

3.3 Specific Requirements

The following are the specific hardware and software requirements of our proposed system. Next we mention the user interface requirements of our proposed QA system.

3.3.1 Hardware Requirements

1. CPU: 2 GHz or faster
2. GPU: Nvidia GPU with 16 GB of memory or higher
3. RAM: 8GB or higher
4. Storage: 100GB or higher, SSD preferred over HDD
5. Processor: A multi-core processor, such as an Intel Core i5 or AMD Ryzen 5

3.3.2 Software Requirements

1. Operating System : windows 7 and above or Linux based OS or MAC OS
2. Programming Language : Python 3.7 or higher
3. Pytorch 2.0 or higher
4. Keras 2.0 or higher

3.3.3 User Interface

1. In order to provide a interactive user friendly interface for the system we use open-source Gradio application.
2. The Gradio interface host our deep learning system and generates a public URL which can be run on any browser for demonstration purpose.

3.4 Tools and Technologies involved

1. Google Colab: Google Colab is a product from Google Research. Colab allows anybody to write and execute arbitrary python code through the browser, and is especially well suited to machine learning, data analysis and education.
2. Python: Python is a general purpose computer programming language often used to build websites and software, automate tasks, and conduct data analysis.
3. Pytorch: PyTorch is an open source machine learning framework based on the Python programming language and the Torch library.
4. Keras: Keras is an open-source library that provides a Python interface for artificial neural networks.
5. Huggingface: Hugging Face is an open-source website that develops tools for building applications using machine learning.
6. Numpy: Numpy is a python library used for scientific computing. It also has functions for working in domain of linear algebra, fourier transform, and matrices.
7. Github: GitHub is a platform and cloud-based service for software development and version control using Git, allowing developers to store and manage their code.
8. Gradio: Gradio is an open-source Python library that is used to build machine learning and data science demos and web applications.

3.5 Advantages offered by proposed system

Proposed system offer many advantages which results in increased time and computational efficiency. Reduction in cost and carbon footprint, increase in performance and accuracy of the models as well as providing multilingual support and easy domain adaptation.

1. Bidirectionality: BERT introduces bidirectional training approach unlike the precursor models, which reads the context in both directions and hence helps in deeper understanding of the language context.
2. Transfer Learning: BERT is based on transfer learning, hence uses pretraining and finetuning approach. Transfer learning is better than training a model from scratch which costs in terms of time and computation efficiency. Transfer learning also significantly reduces the amount of carbon footprint generated by the system.
3. Long Distance Context: BERT uses the Transformer architecture which is solely based on attention mechanism. The Attention mechanism overcomes long-range dependencies problem and support parallel computation which boosts models performance and give state-of-art results.
4. Domain Adaptation: BERT is pretrained on a large, diverse dataset, this allows the model to learn general features that are applicable across different domains. It overcomes the limitation of previous task-specific model structures. BERT can be finetuned on various NLP tasks which include sentimental analysis, text classification, document classification, name-entity recognition etc.
5. Multilingual support: Some variants of BERT models like XLM-RoBERTa are specifically trained on large amounts of unlabelled data which have textual data in different languages. This helps the model learn a general understanding about different languages and it does not need to be trained separately for every language.

Chapter 4

Design

In this chapter we have shown the flowchart and diagrams of our proposed systems mentioned in the last chapter. Next we have shown the overall architecture of BERT models used in the project and provided a detailed description about all the architectural components of the model. Further we have also shown the input output representations of the model for the specific QA task.

4.1 System Diagrams

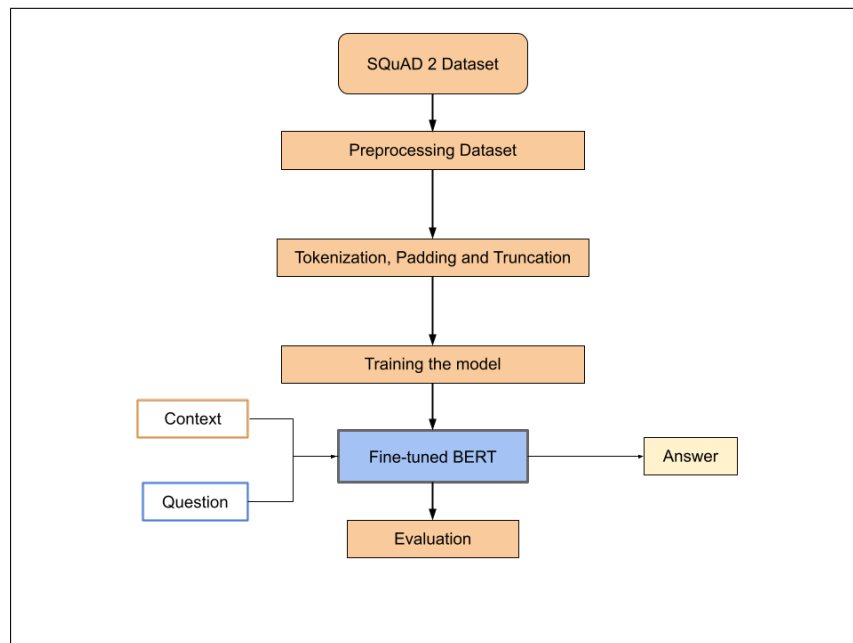


Figure 4.1: Flowchart of Answer-Retrieval QA system

The above flowchart shows the finetuning procedure of BERT-base model on SQuAD 2 dataset in order to train the model for Question-Answering. The other models mentioned in the report are finetuned in the similar fashion. The finetuning procedure includes loading the pretrained model, loading the reading comprehension dataset, preprocessing, training and evaluation of model as shown in the flowchart. The preprocessing steps include tokenizing, padding and truncating the input text. After the training is completed the finetuned BERT model is then ready to retrieve and print the answer from the provided context and question.

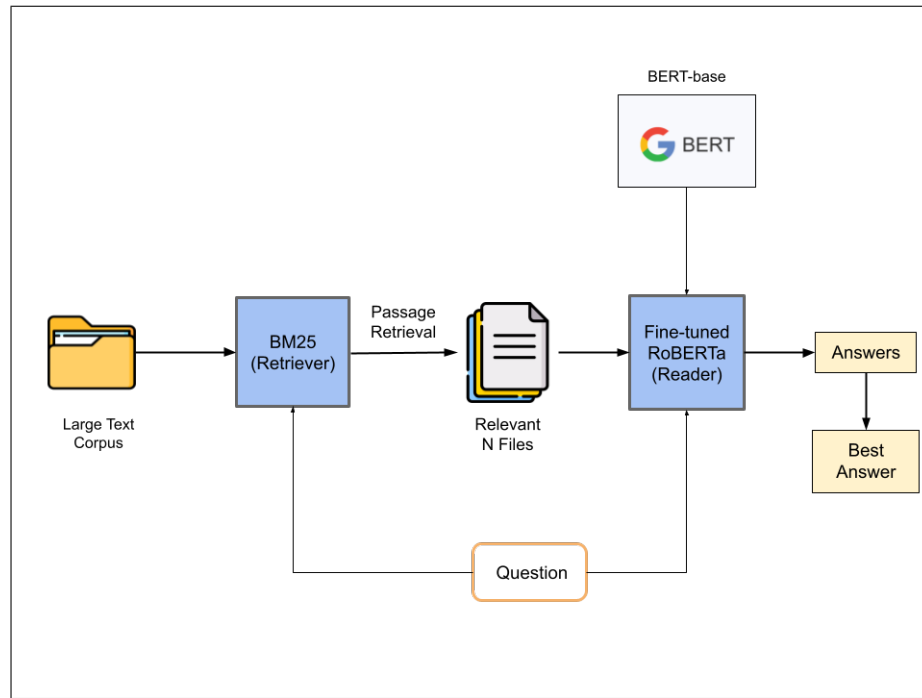


Figure 4.2: Reader-Retriever QA system

Explaining Components of the diagram:

1. Large Text Corpus: The corpus contains multiple files having textual data in an unstructured way. Proposed system has 10 articles files having information about Eye Flu.
2. Question: It refers to the question asked by user.
3. BM25 Algorithm(Retriever): BM25 provides a relevance score to all the text files in the corpus on the basis of the question asked by user. Further it then retrieves 3 text files having the best relevant scores from the corpus.
4. Finetuned RoBERTa (Reader): The model extracts the answer from the retrieved files and prints the answers along with their confidence scores.

-
5. Best Answers: The best answer is selected on the basis of confidence score generated from the RoBERTa model and it is printed to the user.

4.2 BERT Architecture

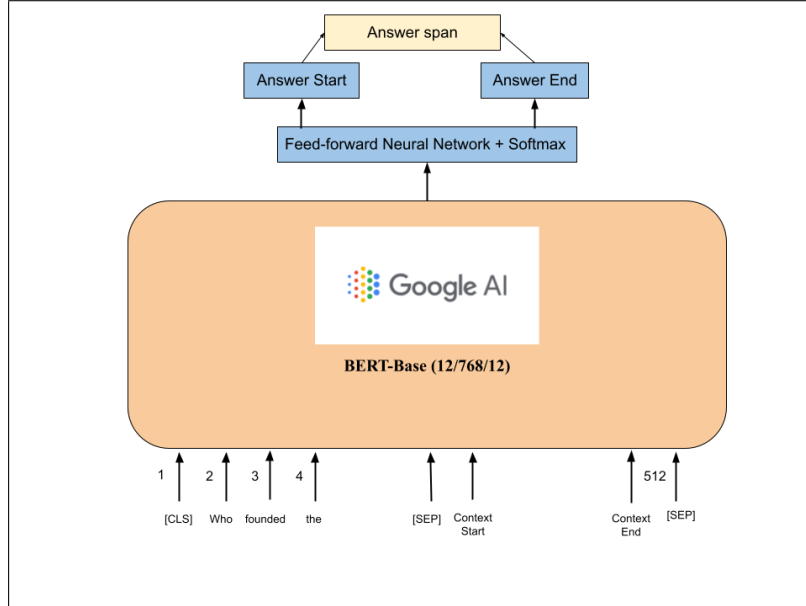


Figure 4.3: Input Output representations of BERT [1]

BERT is basically an Encoder stack of transformer architecture. All the models mentioned in the paper follow the same architecture except for DistilBERT which have 6 encoder layers. They differ in terms of pretraining approach and every model has its unique advantages and strengths. They use the Encoder-part of the Transformer Architecture which is a deep learning architecture based on attention mechanism.

Model architecture is represented in the form of L/H/A where **L** are the number of Encoder layers, **H** is the hidden size and **A** is the number of self-attention heads.

BERT-base (L=12, H=768, A=12, Total Parameters=110M) [1]

Input/Output representations: The input representations which include question and context are packed together in a single sequence after tokenization. The first token of every sequence is always a special classification token ([CLS]). Question is passed after the [CLS] token and it is differentiated from the context using a special separator token ([SEP]). The output representation for question answering involves predicting the start and end positions of the answer span within the passage.

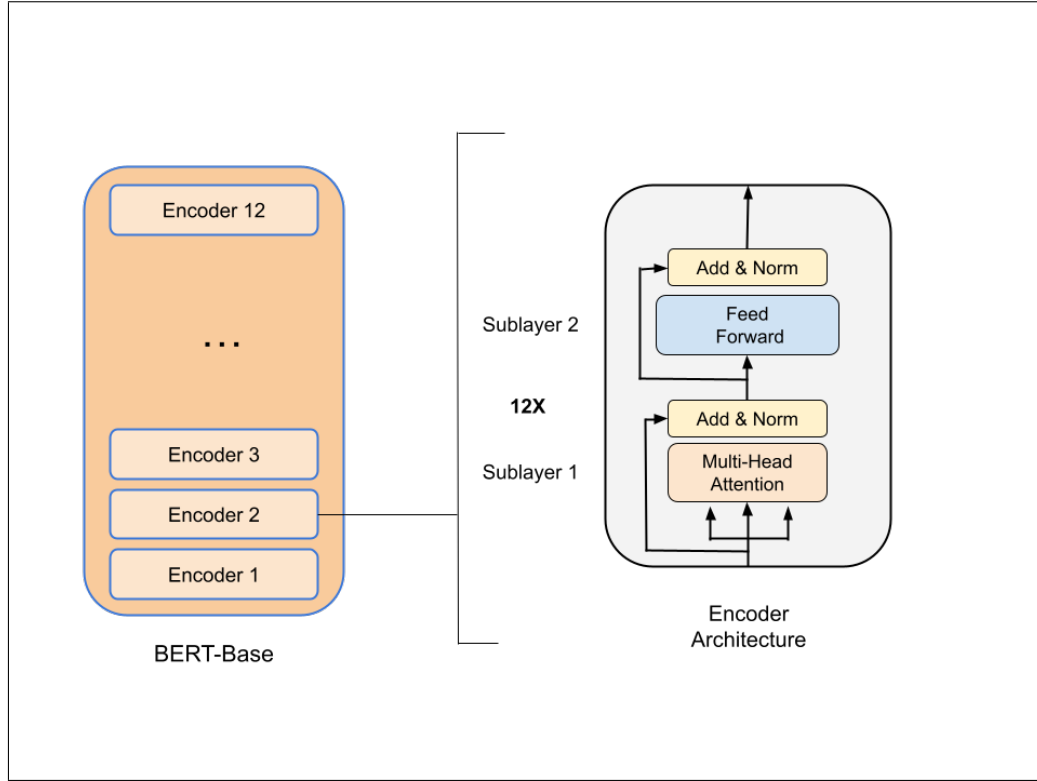


Figure 4.4: Internal Architecture of BERT [2]

As represented, BERT-base has 12 encoder layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position wise fully connected feed-forward network [2]. We employ a residual connection around each of the two sub-layers, followed by layer normalization.

Multi-head attention layer: The encoder's inputs first flow through multi-head attention layer that consists of several self-attention layers running in parallel. The attention mechanism allows the model to focus/or pay attention on specific parts of the input data that are important to the current prediction or decision. It generates attention vectors for all words in the sequence before passing the embeddings to the model.

Feed-forward neural network: The outputs of the multi-head attention layer are fed to this layer. The exact same feed-forward network is independently applied to each position. Both the self-attention and feed-forward sub-layers are followed by layer normalization and residual connections. Layer normalization helps stabilize the learning process, and residual connections mitigate the vanishing gradient problem and enable efficient training.

Chapter 5

Implementation

In this chapter we have shown step by step implementation of proposed system objectives which include dataset details, resources used, preprocessing , training and at the end evaluation of the models. For each objective we have shown the working in different subsections of the chapter.

5.1 Answer-Retrieval QA system

In the implementation section we focus on implementing the objective of the proposed system which includes building a QA system using BERT, RoBERTa, DistilBERT and ALBERT respectively. For this purpose we need to finetune the model on a large QA dataset in the required format. Fine-tuning is the second step in transfer learning. The main goal of transfer learning is to transfer knowledge acquired by training on one task to another task. Fine-tuning, also referred to as domain adaptation, is a process in which the pretrained model is trained with a limited labelled task-specific dataset.

We aim to build the system by first preprocessing the training dataset, then training the model on the processed data using optimized hyperparameters and then evaluating the finetuned model on validation/test dataset. Following are the implementation details of the project:

Model used: BERT-base, RoBERTa-base, DistilBERT and ALBERT-base

Training Dataset used : Training-set SQuAD 2

Evaluation Dataset used: Validation-set SQuAD2

Language: English

Dataset Details:

SQuAD 2 [8] stands for Stanford Question Answering Dataset 2 (SQuAD 2). It is an update to the original SQuAD dataset, which was published in 2016. It is a dataset of reading comprehension questions and answer pairs derived from Wikipedia articles. The answers to all of the questions are provided in the form of a text span. The Question Answering model is trained on the train dataset and evaluated on the validation dataset.

Each example includes the following entries:

`['id', 'title', 'context', 'question', 'answers']`

SQuAD dataset consists of 2 parts:

Train dataset - 130319 examples

Validation dataset - 11873 examples

Resources used:

Python Notebook: Google Colaboratory

Hardware Accelerator used: GPU Tesla T4 provided by google colab

RAM used: 12.78 GB

Disk space used: 78 GB

5.1.1 Preprocessing

The preprocessing is done using the tokenizer, every model has its own corresponding tokenizer associated with it. After preprocessing we create a preprocessing function which configures the preprocessing and code for answer retrieval from the context provided.

1. Loading the Tokenizer: For this we need to import the Autotokenizer module of the transformer library and load the tokenizer using the Autotokenizer.from_pretrained command. The tokenization used is wordpiece tokenization and byte-pair encoding tokenization which have a vocabulary of 30K and 50K respectively.
2. The Tokenizer function: Passing text to the tokenizer function encodes the text into tensors. First the text is divided into subsequent tokens, each of the token is allotted with a input.id and a attention.mask. Special tokens like [CLS] and [SEP] tokens are inserted in the sequence and input-ids are then converted into tensors.
3. Preprocessing hyperparameters initialized include maximum length of the given input sequence (max-length), and the allowed overlap between two input sequences(doc-stride):

-
- (i) max-length = 384
 - (ii) doc-stride = 128
4. Preprocessing parameters: The tokenizer function has 3 important parameters for preprocessing which include padding, truncation, return.tensors. If we want to pass input to the model the inputs should be in a uniform shape. Padding and Truncation thus helps us to create rectangular tensors from batches of varying lengths. Return tensors return the actual tensors fed to the model. The other 2 parameters are return overflowing tokens and offset mappings.
 5. Creating the preprocessing function and Answer Retrieval: The preprocessing function configure all the preprocessing steps into one function which is then applied to the entire dataset using dataset.map function. In order to extract the answers from the context we create a program which will iterate through the whole context and return the start position and end position labels of the answer which will be the inputs.ids of the corresponding words in the context. If the answer is not present in the context the model simply returns a classification token. Preprocessing is applied on the entire dataset using the datasets map function in which by default batch size is 1000.

5.1.2 Training

1. Loading the Model using Autoclass: Training the model starts by importing the required model using AutoModelForQuestionAnswering module and downloading the model using the .from_pretrained() command.
2. Hyperparameter Optimization: BERT model finetuning requires setting many hyperparameter values before training which includes epochs, learning rate, batch size and weight decay. These hyperparameter values have an impact on the models performance so we need to find the optimal set of hyperparameters which gives best results.

The hyperparameter optimization process included:

- (i) Defining a search space and choose initial set of hyperparameters
- (ii) Evaluate model's performance on each set of hyperparameters
- (iii) Run a exhaustive search method for all sets
- (iv) Select the best set of hyperparameters based on the results

After performing hyperparameter optimization we choose the best set of parameters considering the resource limitations and evaluation results. The hyperparameter value used for training are:

-
- Learning rate = $3e-5$
 - Batch size = 16
 - Epochs = 3
 - Weight decay = 0.01
3. **Passing Training Arguments:** In training arguments we pass all the hyperparameter values, training settings which include evaluation and save strategy and the name of the model to be saved.
 4. **Using the Data Collator to batch processed examples before passing to the Trainer():** The Data Collator prepares the processed data from the map function for training. Data collator is a utility that takes care of batching, padding, and collating the input data to format it properly before passing the data for training.
 5. **Configuring and compiling the Trainer:** Trainer function configures all the things required for training which includes model, training arguments, processed training and validation data and the tokenizer.
 6. **Start training and saving the model:** We start training using `trainer.train` method and save the model at completion using `trainer.save` method. After starting the training we can see its progress using a progress bar which shows the number of training steps completed, time taken and it also shows the training loss and validation loss at each step.

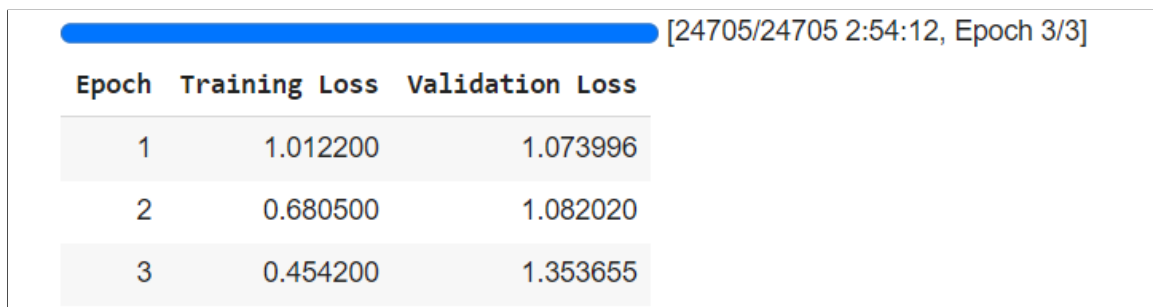


Figure 5.1: Completion of Training of BERT

5.1.3 Evaluation

The 2 most prominent metrics used for evaluating QA systems are :

1. Exact-Match(EM)
2. F1-score

In order to evaluate the fine-tuned models we use the evaluate library and the ipython-autotime library. For the same purpose we import the evaluator module and pass the validation dataset and model checkpoint to the task.evaluator.compute function. The function takes few minutes to iterate through the dataset. It iterates through the validation dataset and calculates the EM and F1-score of each question answer pair, total score is then computed by averaging over the individual example scores.

5.2 Multilingual QA system

To build a multilingual question answering system we make use of XLM-RoBERTa which is a variant of RoBERTa which is pretrained on 2.5 TB of Filtered Common Crawl corpus data. It is pretrained on a diverse multilingual corpus using a combination of two pretraining objectives: MLM and TLM. Following are the implementation details of the project:

Model Used: XLM-RoBERTa-base

Training Dataset used : Training-set SQuAD 2

Evaluation Dataset used: XQuAD

Language: Multilingual (English, Hindi, Spanish etc.)

Evaluation dataset details:

XQuAD [9] stands for Cross-lingual Question Answering Dataset. This dataset is used as a standard tool to evaluate performance in answering cross-lingual questions. A subset of 240 paragraphs and 1190 question-answer pairs from the SQuAD v1.1 development set (Rajpurkar et al., 2016) make up the dataset, together with its expert translations into 10 languages: Hindi, Chinese, Vietnamese, Thai, German, Greek, Russian, Turkish, Arabic, and Spanish. And thus, the dataset is completely parallel in eleven different languages.

Resources used:

Python Notebook: Google Colaboratory

Hardware Accelerator used: GPU Tesla T4 provided by google colab

RAM used: 12.78 GB

Disk space used: 78 GB

The finetuning procedure and evaluation of XLM-RoBERTa is the same as that of objective 1.

5.3 Reader-Retriever QA system

To leverage the performance and use of BERT models we come up with an innovative idea for building a large QA system by adding a retriever (BM25) to the original finetuned RoBERTa model which works as a reader.

Model Used: RoBERTa-base-finetuned-SQuAD2

Retriever: BM25

Resources used:

Python Notebook: Google Colaboratory

Hardware Accelerator used: CPU

RAM used: 12.78 GB

Disk space used: 78 GB

We have shown the working of the proposed system in the design chapter, we further work on implementing it step-wise:

1. Corpus and question: We select a domain for the QA system and create a corpus of multiple txt files having data relating to the domain. Next we ask a question relating to the mentioned domain and store it.
2. Retriever: After loading the files, the code creates an index using the BM25Okapi algorithm from the rank_bm25 library. This index is used to efficiently rank and retrieve relevant files based on their similarity to the input question. The BM25 retriever uses the created index and the input question to rank the passages based on their relevance to the question. It calculates a score for each passage indicating how well it matches the question.
3. Reader: We load the fine-tuned RoBERTa-based QA model and tokenizer associated with it. Using the pipeline function we create an answer retrieval question answering pipeline passing the model and tokenizer to it. The model here works as a reader model.
4. Extracting answers from passages: We now extract the answer using Reader from the files shortlisted by Retriever and calculate the score of each answer.
5. Choosing the best answer: The code selects the answer with the highest confidence score from the extracted answers in the previous step using python max function. This answer is considered the best answer and printed based on the retrieval and extraction process.

Chapter 6

Testing and Experimental Results

In Chapter 6 we show the experimental results of implementation done in the last chapter. We demonstrate comparative analysis of evaluation results of models using tables and charts. In the further subsection we show the evaluation results of our multilingual QA system on 11 languages and illustrate its working. We also illustrate the working of Reader-Retriever QA system.

6.1 Answer-Retrieval QA system

In this section we show the experimental results of the built QA systems using the proposed BERT models and investigate their performance using Training metrics and Evaluation metrics. We draw inference on the basis of results and demonstrate working of the model.

6.1.1 Evaluation of Training Results

Training of Models is done on GPU and it takes a significant amount of time. The training runs almost for 3 epochs over 24000 steps in 180 minutes. For every 500 steps Training Loss and Validation Loss is calculated by the trainer and parameters are updated to reduce the loss value. We note down the loss value for every 1000 steps and make the Training learning curve for each model. Based on the training time, steps and loss value of the models we get the following chart and table.

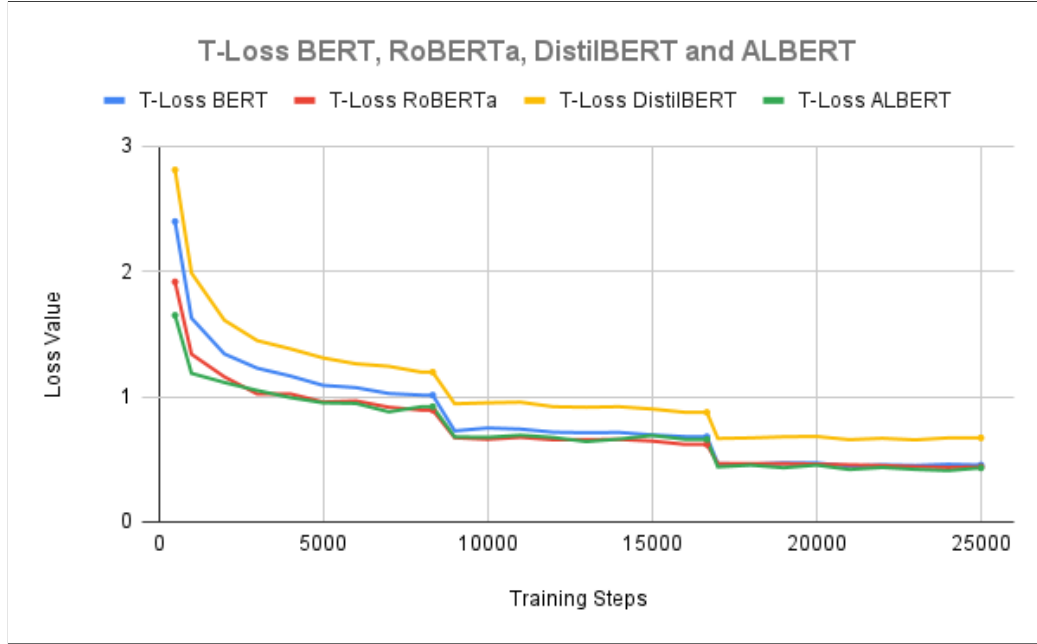


Figure 6.1: Training learning curve of Models

Models	BERT	RoBERTa	DistilBERT	ALBERT
Training Time	2:54:12	3:00:06	1:30:41	3:37:20
Training Steps	24705	24717	24705	24744
Training Loss	0.4542	0.4393	0.6708	0.4314
Validation Loss	1.3537	0.9785	1.4648	0.9899

Table 6.1: Training results of Models

6.1.2 Evaluation Results

As the name implies, EM assigns a score of 1 in the case that every character in the predicted and actual output matches, else it assigns a score of 0. An EM score of zero is given if the actual output differs by just one character. For classification problems, the F1-score is used. It is calculated by first calculating precision and recall and then applying the formula, harmonic mean of precision and recall.

Each question-answer pair’s EM and F1-score are calculated individually, and the model’s overall score is determined by averaging the results of each individual example. We use SQuAD 2’s validation dataset to carry out the evaluation.

Model	BERT	RoBERTa	DistilBERT	ALBERT
EM	73.50	79.72	65.88	78.13
F1	76.79	83.05	68.98	81.55

Table 6.2: Evaluation Results of Models

6.1.3 Inference

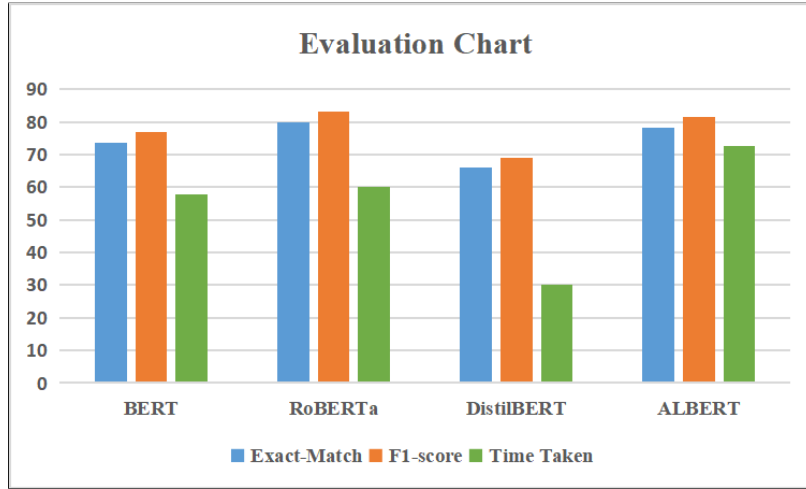


Figure 6.2: Evaluation Chart of Models

From the above performance results we can conclude that:

1. **RoBERTa** [3] gives the highest EM(79.72) and F1-score(83.05) hence it can be used if we want the best prediction results.
2. If we want faster finetuning we can use **DistilBERT** [4]. It has finetuning time almost 50 percent less than that of BERT, although it reduces the performance by some percent.
3. **ALBERT** [5] is computationally inexpensive and also requires less memory to run and it gives score(81.55) almost equal to that of RoBERTa.
4. If we do not need any performance advantages given by BERT variants, using the foundation model **BERT** [1] (76.79) can be a good choice.

6.1.4 Working of Model

As described in Chapter 4 the model is trained to take a question and context as input and it retrieves the answer labels from context and displays the answer.

Context = Gir National Park Wikipedia
Question = What was the population of lions in gir national park in 2015?
Answer = 523

The screenshot shows a web-based interface for an Answer Retrieval QA system. On the left, there is a 'Context Paragraph' section containing two paragraphs of text about Gir National Park and Wildlife Sanctuary. Below this is a 'Question' section with a text input field containing the question 'What was the population of lions in gir national park in 2015?'. At the bottom of the left section are two buttons: 'Clear' and 'Submit'. On the right, there is an 'Answer' section with a text input field containing the answer '523'. Below the answer field is a 'Flag' button.

Figure 6.3: Working of Answer Retrieval QA system

6.2 Multilingual QA system

In this subsection we show the evaluation results of our system on SQuAD 2 and XQuAD dataset. Next we demonstrate the working of model in Hindi, Spanish and German languages.

6.2.1 Evaluation results on SQuAD2 and XQuAD

XLNet-RoBERTa finetuned on SQuAD2 gives the following results:

Training Time = 3:27:28

Training Steps = 24651

Training Loss = 0.5918

Validation Loss = 0.9802

EM = 75.52

F1-score = 78.73

Finetuned XLNet-RoBERTa is adept enough to perform answer retrieval question answering in many languages. Although the results may seem poor, the model gives optimal performance as compared to other existing models.

Language	EM	F1
Arabic (ar)	31.76	46.24
German (de)	52.43	65.02
Greek (el)	46.30	60.99
English (en)	65.12	75.17
Spanish (es)	52.94	70.99
Hindi (hi)	43.95	58.77
Romanian (ro)	56.72	68.81
Russian (ru)	52.01	65.43
Thai (th)	49.57	57.94
Turkish (tr)	47.47	62.27
Vietnamese (vi)	48.57	65.68
Chinese (zh)	39.74	47.70

Table 6.3: Evaluation Results on XQuAD dataset

6.2.2 Working of Model

Context = Delhi city wikipedia

Question = What is the population of Delhi?

Answer = 16.8 million.[4]

Question = Delhi ki jansankya kitni hai?

Answer = 16,8 million

Context Paragraph

दिल्ली, (ए) अधिकारिक तौर पर दिल्ली का राष्ट्रीय राजधानी क्षेत्र (एनसीटी), भारत का एक शहर और केंद्र शासित प्रदेश है जिसमें भारत की राजधानी नई दिल्ली शामिल है। यमुना नदी, मुख्य रूप से इसके पश्चिमी या दाहिने किनारे पर फैली दिल्ली की सीमाएँ पूर्व में उत्तर प्रदेश राज्य और शेष दिशाओं में हरियाणा राज्य के साथ लगती हैं। एनसीटी का क्षेत्रफल 1,484 वर्ग किलोमीटर (573 वर्ग मील) है। [2] 2011 की जनगणना के अनुसार, दिल्ली की जनसंख्या 11 मिलियन से अधिक थी, [3] [12] जबकि एनसीटी की जनसंख्या लगभग 16.8 मिलियन थी। [4] दिल्ली का शहरी समूह, जिसमें राष्ट्रीय राजधानी क्षेत्र (एनसीआर) के नाम से जाने वाले क्षेत्र में उपग्रह शहर गाजियाबाद, फरीदाबाद, गुरुग्रांव और नोएडा शामिल हैं, की अनुमानित आबादी 28 मिलियन से अधिक है, जो इसे भारत का सबसे बड़ा महानगरीय क्षेत्र और दूसरा-बनाता है। दुनिया में सबसे बड़ा (टोक्यो के बाद)।

Question

राष्ट्रीय राजधानी क्षेत्र की जनसंख्या कितनी है?

Clear Submit

Answer

16.8 मिलियन

Flag

Figure 6.4: Demonstration of Hindi QA

Question = Cuál es la población de Delhi?

Answer = 16,8 millones.[4]

Context Paragraph

Spanish: Delhi, [a] oficialmente el Territorio de la Capital Nacional (NCT) de Delhi, es una ciudad y un territorio de la unión de la India que contiene Nueva Delhi, la capital de la India. A caballo entre el río Yamuna, principalmente en su orilla occidental o derecha, Delhi comparte fronteras con el estado de Uttar Pradesh en el este y con el estado de Haryana en las direcciones restantes. El NCT cubre un área de 1484 kilómetros cuadrados (573 millas cuadradas).[2] Según el censo de 2011, la población propiamente dicha de la ciudad de Delhi superaba los 11 millones.[3][12] mientras que la población del NCT era de aproximadamente 16,8 millones.[4] La aglomeración urbana de Delhi, que incluye las ciudades satélite Ghaziabad, Faridabad, Gurgaon y Noida en un área conocida como la Región de la Capital Nacional (NCR), tiene una población estimada de más de 28 millones, lo que la convierte en el área metropolitana más grande de la India y la segunda. más grande del mundo (después de Tokio).

Question

¿Cuál es la población de Delhi?

Clear Submit

Answer

16,8 millones.[4]

Flag

Figure 6.5: Demonstration of Spanish QA

Question = Wie viele Einwohner hat Delhi?
 Answer = 28 Millionen

Context Paragraph

German: Delhi,[a] offiziell das National Capital Territory (NCT) von Delhi, ist eine Stadt und ein Unionsterritorium Indiens mit Neu-Delhi, der Hauptstadt Indiens. Delhi liegt am Yamuna-Fluss, hauptsächlich am westlichen oder rechten Ufer, und grenzt im Osten an den Bundesstaat Uttar Pradesh und in den übrigen Richtungen an den Bundesstaat Haryana. Das NCT umfasst eine Fläche von 1.484 Quadratkilometern (573 Quadratmeilen).[2] Laut der Volkszählung von 2011 betrug die Einwohnerzahl der Stadt Delhi über 11 Millionen.[3][12] während die Bevölkerung des NCT etwa 16,8 Millionen betrug.[4] Delhis städtische Agglomeration, zu der die Satellitenstädte Ghaziabad, Faridabad, Gurgaon und Noida in einem Gebiet gehören, das als National Capital Region (NCR) bekannt ist, hat eine geschätzte Bevölkerung von über 28 Millionen und ist damit die größte Metropolregion Indiens und die zweitgrößte. größte der Welt (nach Tokio).

Question

Wie viele Einwohner hat Delhi?

Clear Submit

Answer

28 Millionen

Flag

Figure 6.6: Demonstration of German QA

6.3 Reader-Retriever QA system

In this subsection we show the selection of problem domain and formation of text corpus of our QA system. Next we demonstrate the question asked to our system, all answers displayed, and the best answer displayed according to the best corresponding confidence score.

6.3.1 Domain and Text Corpus

We need to select a Problem Domain for our QA system. The domain refers to the specific subject area or topic that the question answering system focuses on, it defines the scope of the system and the types of questions it is designed to answer. In this case we select "**Eye Flu**" from medical domain for demonstration.

Creating Document store: In order to create a document store on the selected domain we create a text corpus containing 10 txt articles written on eye flu. Following are the articles included in the document store:

1. Article 1 - Navigating Eye Flu-Symptoms and Remedies
2. Article 2 - Eye Flu and Its Impact on Daily Activities
3. Article 3 - The Intricacies of Eye Flu - Unveiling Symptoms and Management
4. Article 4 - Eye Flu - Navigating the Discomfort
5. Article 5 - Eye Flu - More Than Meets the Eye
6. Article 6 - The Impact of Eye Flu on Public Health
7. Article 7 - Navigating the Symptoms and Treatment of Eye Flu
8. Article 8 - The Science Behind Eye Flu Causes and Contagion
9. Article 9 - The Seasonal Surge Exploring the Relationship Between Eye Flu and Allergies
10. Article 10 - Emerging Trends in Conjunctivitis Research From Genetics to Treatment

6.3.2 Working of QA system

The model outputs a confident score for every answer. The higher the score- the greater the confidence in the answer. Based on the confident scores the code selects the top n answer for the given question by the user. From the shortlisted answers code selects the best answer on the basis of scores and displays it along with context and score.

Question asked by user: What are the medicines for eye flu?

Question: What are the medicines for eye flu?

All Answers:
Answer: Over-the-counter antihistamine eye drops
Context: Eye flu, or conjunctivitis, is a condition that shares symptoms with allergic conjunctivitis,
Score: 0.0005313202273100615

Answer: Biologics and immunomodulatory drugs
Context: Eye flu, medically referred to as conjunctivitis, has been a subject of scientific exploration
Allergic conjunctivitis research is focusing on understanding the underlying immune response and identi
Score: 0.00039075600216165185

Answer: allergen avoidance and antihistamine
Context: Eye flu, scientifically known as conjunctivitis, is a condition that affects the conjunctiva,
Score: 0.0002973204245790839

Figure 6.7: All answers output by the model

Best Answer:
Question: What are the medicines for eye flu?
Answer: Over-the-counter antihistamine eye drops
Context: Eye flu, or conjunctivitis, is a condition that shares symptoms with allergic conjunctivitis,
Score: 0.0005313202273100615

Figure 6.8: Best answer among all output answers

Chapter 7

Conclusion & Future Work

In this project report we have studied about Encoder-based LLMs and their working. We have utilized the LLMs to build Answer-Retrieval QA systems using a reading comprehension dataset (SQuAD 2). We have further evaluated their performance and demonstrated its working. Next we have done a comparative analysis of their results and then shown inference on the basis of results using tables and charts. Further we have successfully built a multilingual QA system using XLM-RoBERTa and evaluated its performance on XQuAD dataset which consists of 11 languages. We have illustrated its working across Hindi, Spanish and German languages. In the final objective we have augmented our QA system by adding a retriever to our Answer Retrieval QA system and hence building a Reader-Retrieval QA system. We have demonstrated its working on a manually created corpus and illustrated its results in experimental results.

Future work for BERT-based models include reducing dependency on hardware accelerators like GPUs and TPUs. This includes either working towards making hardware accelerators open-source or reducing the total parameters of the encoder model while maintaining its efficiency and performance. Research can be done on exploring new architecture for the models and further hyperparameter tuning which will result in better performance. Although using BERT reduces the carbon footprint because of transfer learning we can still improve further in terms of minimizing the carbon footprint.

References

- [1] Devlin, J., Chang, M., Lee, K., Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv (Cornell University). <https://arxiv.org/pdf/1810.04805v2>
- [2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). Attention is All you Need. arXiv (Cornell University), 30, 5998–6008. <https://arxiv.org/pdf/1706.03762v5>
- [3] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V. (2019). ROBERTA: A robustly optimized BERT pretraining approach. arXiv (Cornell University). <https://doi.org/10.48550/arxiv.1907.11692>
- [4] Sanh, V., Debut, L., Chaumond, J., Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv (Cornell University). <https://arxiv.org/pdf/1910.01108.pdf>
- [5] Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., Soricut, R. (2019). ALBERT: A lite BERT for self-supervised learning of language representations. arXiv (Cornell University). <https://doi.org/10.48550/arxiv.1909.11942>
- [6] Conneau, A. (2019, November 5). Unsupervised cross-lingual representation learning at scale. arXiv.org. <https://arxiv.org/abs/1911.02116>
- [7] Wang, Z. (2022, June 30). Modern Question answering Datasets and Benchmarks: A survey. arXiv.org. <https://arxiv.org/abs/2206.15030>
- [8] The Stanford question answering Dataset. (n.d.). <https://rajpurkar.github.io/SQuAD-explorer/>
- [9] xquad. (n.d.). TensorFlow. <https://www.tensorflow.org/datasets/catalog/xquad>
- [10] Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389. <https://doi.org/10.1561/15000000019>

-
- [11] Hambarde, K. A. (2023, January 20). Information retrieval: recent advances and beyond. arXiv.org. <https://arxiv.org/abs/2301.08801>
 - [12] Alammar, J. (n.d.-b). The illustrated transformer. <https://jalammar.github.io/illustrated-transformer/>
 - [13] Alammar, J. (n.d.-a). The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning). <https://jalammar.github.io/illustrated-bert/>
 - [14] Transformers. (n.d.). <https://huggingface.co/docs/transformers/index>
 - [15] Evaluate. (n.d.). <https://huggingface.co/docs/evaluate/index>
 - [16] Biswal, A. (2023). Recurrent Neural Network(RNN) tutorial: types, examples, LSTM and more. Simplilearn.com. https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn#how_does_recurrent_neural_networks_work
 - [17] Team, G. (n.d.). Gradio. <https://www.gradio.app/>
 - [18] Weng, L. (2020, October 29). How to build an Open-Domain question answering system? Lil'Log. <https://lilianweng.github.io/posts/2020-10-29-odqa/>
 - [19] GeeksforGeeks. (2023). Deep Learning tutorial. GeeksforGeeks. <https://www.geeksforgeeks.org/deep-learning-tutorial/>
 - [20] rank-bm25. (2022, February 16). PyPI. <https://pypi.org/project/rank-bm25/>
 - [21] Connelly, S., Connelly, S. (2019a). Practical BM25 - Part 1: How shards affect relevance scoring in Elasticsearch. Elastic Blog. <https://www.elastic.co/blog/practical-bm25-part-1-how-shards-affect-relevance-scoring-in-elasticsearch>

Document Information

Analyzed document	UI19CO48_Final Report.pdf (D173702459)
Submitted	2023-09-11 12:46:00
Submitted by	
Submitter email	library@iiitsurat.ac.in
Similarity	5%
Analysis address	library.iiitsr@analysis.orkund.com

Sources included in the report

SA	IITB_MTP_Dissertation_Sanyam_.pdf Document IITB_MTP_Dissertation__Sanyam__.pdf (D110154562)	 2
SA	Raveen.pdf Document Raveen.pdf (D154030976)	 2
W	URL: https://arxiv.org/pdf/1810.04805v2 Fetched: 2023-09-11 12:46:00	 2
W	URL: https://arxiv.org/pdf/1910.01108.pdf Fetched: 2023-09-11 12:46:00	 1
SA	R00195785_Renuka_Patil+%28%29.pdf Document R00195785_Renuka_Patil+%28%29.pdf (D136656724)	 1
SA	M1.888_20212_PEC 2. Entrega de seguimiento_17068494.txt Document M1.888_20212_PEC 2. Entrega de seguimiento_17068494.txt (D132461125)	 1
W	URL: https://arxiv.org/pdf/1706.03762v5 Fetched: 2023-09-11 12:46:00	 2
W	URL: https://jalammar.github.io/illustrated-transformer/ Fetched: 2023-09-11 12:46:00	 1
W	URL: https://www.tensorflow.org/datasets/catalog/xquad Fetched: 2023-09-11 12:46:00	 1
SA	Team9_Analysis_of_Language_Models.pdf Document Team9_Analysis_of_Language_Models.pdf (D103215965)	 3
SA	revised_crosslingual_ajcc 28-03-2023.docx Document revised_crosslingual_ajcc 28-03-2023.docx (D162512418)	 1
SA	crosslingual_ajcc.pdf Document crosslingual_ajcc.pdf (D159377708)	 1